

021 指标拆分与可信评价实验

目标是把一个评价协议完整执行一遍：用独立手算核对指标，从验证数据选择阈值，冻结后读取测试数据，并说明有限结果能支持什么。本实验只用原创合成记录，不调用网络、付费 API 或深度学习框架。

1 文件与运行前约定

先读 lecture.md 第 1 至 5 节。核心 experiment.py 与 test_experiment.py 只依赖 Python 标准库。实验不需要下载数据、GPU 或随机种子，所有数据均为固定小表。Notebook 需要一个支持 Python 的 Jupyter 前端与 ipykernel。

文件	用途
data/train.csv	8 条训练记录，只用于多数类基线与拆分检查
data/validation.csv	8 条验证记录，用于五个阈值的比较
data/sealed_test.csv	8 条冻结后评价记录，提前查看会破坏练习体验
data/split_panel.csv	独立的 3 实体乘 3 时点拆分练习
experiment.py	指标、拆分、校准分箱与主流程
experiment.ipynb	解释与实际输出交错的顺序实验
test_experiment.py	有理数算术、边界、隔离与复现测试

输入字段为 row_id、entity、time、group、sensor、y。score 由 sensor/10 得到，不从标签反推。时间是相对整数序号，不是真实日期；group 是合成子组 A/B，不含真实人员信息。主数据每实体有两条记录，不能视为 8 个独立实体。

在动手前抄下协议：正类为 1；阈值网格为 0.3、0.4、0.5、0.7、0.9；相等分数判正；漏报成本 3、误报成本 1；总成本并列时选择更高阈值；最终分组 A/B；zero_division=0，并显示 undefined。明确这些约定后，才进行数值比较。

2 在新进程运行脚本

从单元目录运行：

```
python experiment.py
python -m unittest -v test_experiment.py
```

需要另存结果时：

```
python experiment.py --output-dir my_run
```

脚本也支持从其他当前目录用完整脚本路径运行；默认数据位置相对于脚本，不依赖当前目录。输出目录不能是输入数据目录、其后代、其祖先，或单元源码目录及其祖先。已有输出文件若为符号链接，会拒绝写入。该约束用于防止把练习数据覆盖成结果，不是通用安全沙箱。

成功时终端摘要为 `threshold=0.4`，测试计数 `TN=3、FP=2、FN=1、TP=2`，`test_mean_cost=0.625`，`test_f1` 约为 0.5714286。先记下输出，再解释它的来源，不要把“运行无错误”当作正确性的全部证据。

四个输出文件分别是冻结方案 `frozen_plan.json`、完整结果 `summary.json`、验证阈值表 `thresholds.csv` 和独立校准例的 `calibration.csv`。成功重跑会替换四个正式结果；每次先在隔离目录写方案、再读测试、完成计算，输入或计算失败不改变旧结果。这不是操作系统故障下的多文件原子事务。不会修改 CSV 输入；自行修改过结果且想保留时，使用新的输出目录。

3 先算八条记录，再看函数

只打开 `validation.csv`，写出阈值 0.5 的预测序列。逐条分到四个格子，检查四格之和等于 8。计算 `precision`、`recall`、`F1` 和平均成本；然后调用 `binary_metrics` 与 `threshold_table` 交叉核对。

```
from experiment import binary_metrics, classify, threshold_table
truth = [1, 0, 1, 0, 1, 0, 0, 0]
scores = [.9, .8, .7, .6, .4, .3, .2, .1]
print(binary_metrics(truth, classify(scores, .5)))
print(threshold_table(truth, scores))
```

检查两个独立条件：计数是否正确；分母是否对应你的问题。再输入 `y=[0,0]`、`prediction=[0,0]`，记录 `precision`、`recall`、`F1` 的填充值及 `undefined`。不要只检查数值而忽略“没有被评价到”的含义。

练习 A：把相等阈值的比较符号从 $\geq$ 想象为 $>$ ，指出哪一行会改变，解释为什么接口语义必须固定。不要修改主流程后仍把旧测试结果当作同一版本。

4 阈值冻结与读取顺序

查看 `make_plan`：它只接收训练、验证记录和开发数据目录，计算候选成本后返回冻结方案。`run` 在隔离的本次尝试目录将方案写盘后，才调用 `load_rows` 读取 `sealed_test.csv`。`summary` 的 `audit` 显示这一顺序。

单元测试用观察包装器在读取测试文件的瞬间检查方案是否已写入。另一个测试复制数据到临时目录，翻转测试标签并改变测试 `sensor`，重新执行。两个 `frozen_plan.json` 必须逐字节一致，而测试结果必须变化。这检验选择没有依赖测试值，不等于证明一个任意复杂系统绝无泄漏。

练习 B：阅读冻结方案后，在纸上预测当验证阈值已固定为 0.4、测试分数整体降低时，哪些计数可能变化。然后检查实际测试结果，并写一句含分母的结论。不得依据这次结果继续挑阈值。

5 多分类与回归对照

把正文的 3×3 混淆矩阵展开为 13 条真实与预测标签。手算三类 F1，调用 `multiclass_metrics(labels=[0,1,2])`，再加未出现的类 3。分别使用 `zero_division=0` 与 1，解释为什么宏平均改变，而完整单标签微平均仍为 10/13。

回归用 `y=[2,4,6,8]`、`prediction=[3,4,4,9]`，先手算误差、绝对误差与平方误差，再调用 `regression_metrics`。追加常数目标 [5,5] 和预测 [5,5]，确认 R^2 为 null 而非声称测到 1。本实验采用显式缺失约定，不能与其他库的默认替代值直接混比。

回归计算只接受有限数值，绝对值不超过 $1e100$ ；拒绝 bool、NaN、无穷和平方下溢造成的非零误差消失。极端尺度应先换合理单位。教学实现不追求覆盖任意浮点极端条件。

6 分箱和分组需要保留分母

校准例独立于告警主数据。使用正文八个概率和标签，逐个标记所属箱；边界 0.25、0.5、0.75 属于右侧箱，1 属于最后一箱。先预测四个箱的数量，再查看 `calibration_bins` 输出。用两条端点概率 [0,1] 再运行一次，确认中间空箱的均值与频率为 null，未被画成观察值 0。

练习 C：将边界改成 [0,0.5,1]，重算 ECE。结果与原来是否一致？如果不一致，说明原因，而不是宣称其中一个“更真实”。不要在测试上挑使 ECE 最小的边界作为事先固定的协议。

查看 `test_groups`，逐项核查总数、support、`predicted_positive`、`undefined`。A 组两个正例都找到，B 组一个正例没找到。写明为何这既值得继续调查，又不足以给出已确认的人群差异结论。

7 实体与时间拆分

对 `split_panel.csv` 分别运行 `split_by_entity(panel,['A'],['B'],['C'])` 与 `split_by_time(panel,1,2)`。前者实体互斥但时间不严格分离；后者时间严格分离但实体重叠。尝试把时间拆分结果交给默认要求两种隔离的 `check_partitions`，应当报错。

练习 D：若目标是“新机器未来运行”，单独选一种拆分为何不够？说明主实验采取的两种隔离，以及仍未解决的标签延迟、代表性和实体相关性问题的。

8 Notebook 重启顺序执行

建议环境安装方式如下，安装本身可能需要联网获取软件包，数值实验运行时离线：

```
conda env create -f environment.yml
conda activate dl021
jupyter lab
```

打开 `experiment.ipynb`，重启内核后从第一格顺序运行到最后。Notebook 将主流程另存到 `notebook_outputs`，最后核对四份结果与随附 `outputs` 逐字节一致。不要只运行最后一格并依赖旧变量。

作者实际在一个全新 Python 进程中的真实 `ipykernel InProcessKernel` 顺序执行 Notebook，保留所有输出。这核验了内核执行与顺序状态，不包含浏览器界面、跨进程 `socket` 通信、全新 Anaconda 安装或跨操作系统安装测试。

9 交付你的结论

提交手算、冻结方案、四份输出和一段评价文字。文字至少包括任务与正类、拆分单位、样本量和正例数、阈值如何选、测试计数与成本、缺失指标约定、分组描述和不能推广的范围。答案见 `answers.md` / `answers.pdf`。

如果想改成本、指标或模型，可复制一份开发实验继续探索，并把已经看过的测试集视为开发信息；要获得下一轮独立证据，需要新的合适测试样本。不能靠重命名文件恢复独立性。